

Tarea 5

Universidad Nacional Autónoma de México

Aprendizaje por refuerzo

Sandra Alitzel Vázquez Chavez

I. PARETO CONDITIONED NETWORKS

Basándose en la arquitectura de Pareto Conditioned Networks, el algoritmo original para acciones discretas fue adaptado a problemas de control continuo. A diferencia de los métodos tradicionales de actor-crítico, PCN aprende una única red de política que está condicionada a un retorno multiobjetivo deseado. Para su implementación y evaluación, el flujo se estructuró en los siguientes módulos:

- `core.py`: Estructuras de datos base, memoria de trayectorias y ordenamiento de Pareto.
- `networks.py`: Arquitectura del actor condicionado.
- `wrappers.py`: Adaptación de los entornos y vectorización segura.
- `hypervolume.py` y `compare_baselines.py`: Métricas de evaluación y comparativa empírica.

II. PCN CORE

El núcleo algorítmico de PCN se basa en clonación de comportamiento guiado por los retornos multiobjetivo. Se compone fundamentalmente de tres bloques:

- Experience Buffer: A diferencia de los buffers por pasos, este almacena episodios completos. Durante la inserción, calcula el retorno por obtener para cada paso, definido como $G_t = \sum_{k \geq t} r_k$, sin aplicar factor de descuento.
- Non-dominated sorting: Dado el registro de retornos por episodio, extrae los índices de las trayectorias óptimas de pareto asumiendo una maximización para todos los objetivos.
- Muestreo de Retorno Objetivo: Para obligar a la política a superar sus propios records, el algoritmo selecciona un vector de retorno del frente empírico y lo empuja hacia afuera sumándole ruido gaussiano proporcional a la dispersión del frente. El resultado siempre es truncado inferiormente para garantizar que el nuevo comando domine al punto de referencia.

III. CONDITIONEDACTOR

En lugar de tener múltiples políticas, PCN utiliza un ConditionedActor. Este módulo implementa la red neuronal encargada del control continuo:

- Entradas: La red recibe una concatenación del estado actual del entorno, el retorno multiobjetivo deseado y el horizonte de tiempo restante.

- Salidas: saca los parámetros media y desviación estándar de una distribución gaussiana diagonal sobre el espacio de acciones.
- Inicialización: Se emplea una inicialización ortogonal en los pesos de las capas lineales para reducir drásticamente la varianza durante el entrenamiento. La desviación estándar puede configurarse para ser dependiente del estado o como un parámetro global aprendible.

IV. RECOMPENSAS VECTORIALES

Para que PCN pueda optimizar múltiples objetivos simultáneamente, se implementaron adaptaciones específicas sobre el simulador MuJoCo:

- Extracción Multiobjetivo: El entorno MOHalf-CheetahWrapper descompone la recompensa escalar nativa en un vector bidimensional con componentes reward run, reward ctrl.
- Estabilización numérica: Las recompensas son divididas por 100 para mantener ambos objetivos en un rango de magnitud comparable, previniendo así el desbordamiento numérico en el buffer y estabilizando la regresión gaussiana del actor.
- Vectorización aislada: Para la recolección paralela de datos se garantiza que cada trabajador inicie su propio intérprete y su propia instancia de MuJoCo en total aislamiento, evitando fallos de segmentación y fugas de memoria.

V. BASELINE DE COMPARACIÓN

Para evaluar rigurosamente la implementación de PCN, se definió un script comparativo que enfrenta la arquitectura contra PGMORL, un algoritmo multiobjetivo extraído de la librería morl-baselines. Ambos algoritmos son entrenados bajo condiciones iguales, reciben el mismo número de semillas aleatorias independientes y operan sobre directorios aislados para garantizar la idempotencia del benchmark.

VI. EVALUACIÓN

El rendimiento de ambos algoritmos se evalúa analizando los frentes de Pareto empíricos resultantes mediante las siguientes métricas:

- Hipervolumen: Calcula la región dominada por el frente por encima de un punto de referencia fijo

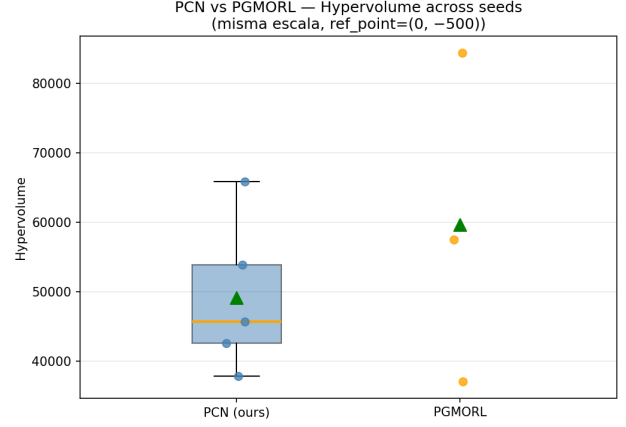
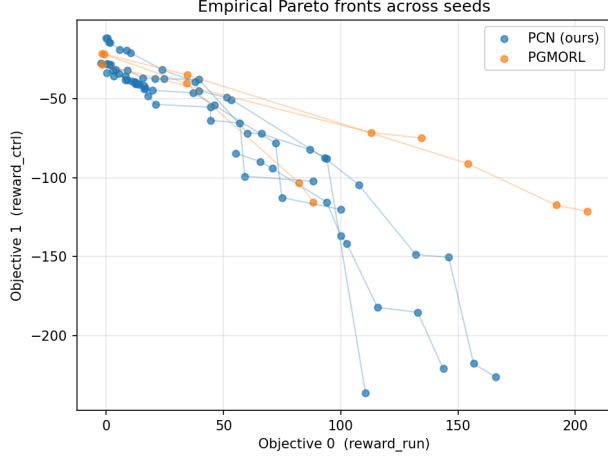


Figura 1. Enter Caption

r . Para esta comparativa, el punto de referencia se unificó a en la escala original del entorno. Antes de calcular el hipervolumen de PCN, su frente se reescala multiplicándolo por 100 para deshacer la estabilización interna y garantizar una comparación justa con PGMORL.

- **Tests Estadísticos:** Se implementa un test de suma de rangos para validar estadísticamente con un nivel de significancia $\alpha = 0,05$, si un algoritmo supera al otro de forma consistente a través de las distintas semillas, descartando que los resultados sean producto del azar.

VII. RESULTADOS

A continuación se presentan los resultados de la evaluación empírica comparando la arquitectura PCN contra el baseline PGMORL, utilizando el mismo punto de referencia (0.0, -500.0) a lo largo de múltiples semillas aleatorias.

VII-A. Frente de pareto

Observando la gráfica de dispersión de los frentes de Pareto empíricos, se identifican dinámicas de exploración fuertemente contrastantes entre ambos métodos:

- **Trade-off y Límites Extremos:** Ambos algoritmos evidencian el conflicto natural del entorno: correr más rápido (Objective 0) cuesta más energía (Objective 1). Sin embargo, PGMORL (naranja) demuestra una mayor capacidad para alcanzar velocidades extremas (cercanas a 200) manteniendo una penalización de control relativamente moderada. PCN (azul), por el contrario, sufre un colapso energético severo (caídas casi verticales en la gráfica) cuando intenta superar la barrera de los 100 o 150 puntos de velocidad.

- **Densidad vs. Extensión:** El comportamiento de PCN destaca por la densidad de sus frentes. Genera una gran cantidad de políticas intermedias bien distribuidas en las zonas de baja y media velocidad, lo que ofrece un abanico mucho más rico de opciones equilibradas. PGMORL, aunque llega más lejos en el eje de la velocidad, produce frentes notablemente escasos y dispersos, encontrando muy pocos puntos intermedios de compromiso.

VIII. HIPERVOLUMEN

Para medir la calidad general de las políticas encontradas, se analizó el hipervolumen de los frentes de pareto descubiertos.

- **Desempeño Promedio:** De acuerdo con el reporte estadístico, PCN completó exitosamente la ejecución en sus 5 semillas, obteniendo un hipervolumen promedio de 49,189.87 con una desviación estándar de 11,004.64. Por su parte, el baseline PGMORL logró resultados en 3 semillas, alcanzando un promedio mayor de 59,662.98, pero presentando una inestabilidad mucho mayor reflejada en su desviación estándar de 23,725.44.
- **Significancia Estadística:** para validar si la superioridad aparente de PGMORL en la media era real, se obtuvo un valor p de 0.6547. Dado que este valor es muy superior al nivel de significancia establecido ($\alpha = 0,05$), no existe evidencia estadística suficiente para afirmar que la diferencia de desempeño sea significativa.

No estoy segura de por qué no se ve la caja del PGMORL, pero lo que se mencionó anteriormente sí se basa en datos de los experimentos.